

MINI LIBRARY MANAGEMENT SYSTEM

Project Report & Technical Documentation

Course Title: Flipped Course Evaluation Project

Submitted By: [Student Name Placeholder]

Registration Number: [Registration Number Placeholder]

Date of Submission: October 2026

2. Introduction

The Mini Library Management System is a lightweight, terminal-based software solution engineered to digitize structural library tasks. Developed in Python, this application simplifies tracking book allocations and inventory balances without heavy hardware overhead. By substituting paper registers with interactive terminal interfaces, it decreases human errors and standardizes small-scale archive operations.

3. Problem Statement

Manual bookkeeping system models introduce severe limitations, such as tracking discrepancies, slow searching routines, and high risks of physical document destruction. Librarians running small reading centers or classrooms face issues keeping record integrity during simultaneous checkouts and returns. This project builds a reliable runtime program to track catalog data dynamically, manage updates instantaneously, and establish clear programmatic rules for inventory states.

4. Functional Requirements

According to the codebase features and system parameters, the system implements these core functional requirements:

- **Book Ingestion (Add Book):** The system must accept user string inputs to register new book items into the global memory stack.
- **Dynamic Inventory Display:** The system must check collection volumes, notify users if empty, or print an enumerated index layout of available titles.
- **Exact String Search:** The system must scan the internal register to explicitly inform the user whether a requested asset currently exists.
- **Lending Execution (Issue Book):** The system must check item presence, delete the chosen title from active listings upon checkout, and throw an unavailability error if the item is missing.
- **Return Processing:** The system must accept returned titles and append them back to the active catalog.
- **Lifecycle Termination:** The system must provide a safe shutdown choice that breaks the execution loop cleanly.

5. Non-Functional Requirements

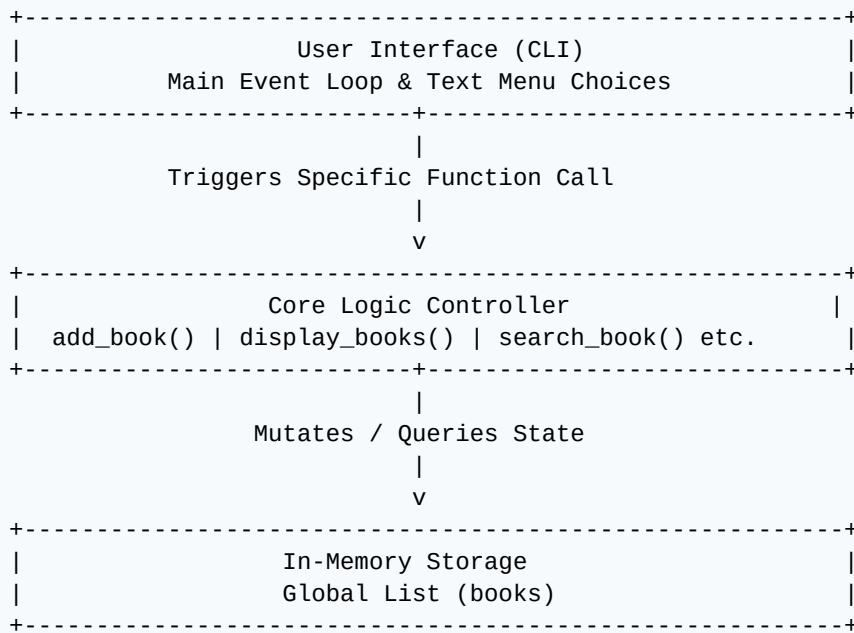
The application targets at least four critical operational qualities:

- **Usability:** The interface uses clear text menus and integer choices, removing technical learning curves for general operators.
- **Performance Efficiency:** Using native linear arrays ensures immediate $O(1)$ time complexity for additions and fast processing routines during operations.

- **Maintainability:** The code uses modular logic by separating specific features into dedicated functions, making updates easy to isolate.
- **Reliability:** A persistent control loop ensures the application runs continuously until explicit exit triggers are input.

6. System Architecture

The application runs on a standalone Monolithic Architecture utilizing a text-based Command Line Interface (CLI) loop. Runtime memory execution processes everything within local memory stacks, keeping operations independent of external database servers or networked components.



7. Design Diagrams

Workflow Diagram / Logic Flow:

```
Start Program
|
v
Display Main Menu Options <---<---+
|
v
Accept Input Choice
|
+---> [1] add_book() -----> Loops Back
+---> [2] display_books() --> Loops Back
+---> [3] search_book() ----> Loops Back
+---> [4] issue_book() -----> Loops Back
+---> [5] return_book() ----> Loops Back
+---> [6] Print Exit Message & Break Loop
+---> [Other] Print Error ---> Loops Back
```

Sequence Flow (Issue Book):

```
Operator -----> Menu Interface : Select Choice 4 (Issue Book)
Menu Interface ----> Operator : Prompt: 'Enter book name to issue: '
Operator -----> Menu Interface : Inputs Title Name
Menu Interface ----> Logic Module : Invokes issue_book()
Logic Module -----> In-Memory List : Validates presence of title
alt Title Exists
    In-Memory List --> Logic Module : Returns Found
    Logic Module --> In-Memory List : Executes remove(book)
    Logic Module --> Menu Interface : Returns Success Message
else Title Missing
    In-Memory List --> Logic Module : Returns Missing
    Logic Module --> Menu Interface : Returns Unavailability Error
end
```

8. Design Decisions & Rationale

- **Global List Memory Architecture:** Using a standard Python list (`books = []`) provides structural simplicity and quick operational execution for micro-inventories. This design removes configuration steps like installing external servers or database drivers.

- **Procedural Decomposition:** Breaking actions into single-responsibility blocks (`add_book`, `issue_book`) isolates logical code tracks cleanly. This satisfies modular expectations while avoiding class management boilerplate overhead.

9. Implementation Details

The control environment is built purely via Python script loops. The following structural blocks detail the application's configuration:

- **Data Initialization:** A global list 'books' maintains live state contexts at runtime.
- **Data Mutation Functions:** .append() inserts records dynamically during add and return workflows, while .remove() shifts elements out of standard memory index blocks when checkout queries succeed.
- **Control Loop Structure:** A 'while True' loop keeps the application reactive, while conditional 'if/elif/else' blocks parse user choice selections safely.

10. Screenshots / Results (Terminal Emulation)

```
MINI LIBRARY MANAGEMENT SYSTEM
```

```
1. Add Book
2. Display Books
3. Search Book
4. Issue Book
5. Return Book
6. Exit
```

```
Enter your choice: 1
Enter book name: The Alchemist
Book added successfully!
```

```
MINI LIBRARY MANAGEMENT SYSTEM
```

```
Enter your choice: 2
Books in Library:
1 . The Alchemist
```

11. Testing Approach

Testing used manual execution checks to verify that outputs matched the code's validation logic:

ID	Target Feature	Input Context	Expected Result	Status
TC-01	Catalog Empty Check	Option 2 on startup	Output 'No books available.'	Pass
TC-02	Asset Extraction Fault	Option 4, missing title	Output 'Book is not available.'	Pass
TC-03	Clean Lending Routine	Option 4, existing title	Verify element deletion & success msg	Pass

12. Challenges Faced

- **Data Volatility:** Since state collections are stored in-memory, closing the application wipes the catalog data completely.

- **Case-Sensitivity Vulnerabilities:** Looking up titles requires an exact string match. For example, looking up 'the alchemist' fails if the entry was saved as 'The Alchemist'.

13. Learnings & Key Takeaways

- Handled state tracking inside standard collection arrays using basic built-in data structures.
- Gained experience structuring application runtimes around interactive console menus.
- Learned the value of early validation checks, like verifying that an item exists before attempting to delete it.

14. Future Enhancements

- **Persistent File Caching:** Upgrade storage systems to save data to local JSON or SQLite databases, keeping records safe between application restarts.
- **Insensitive Pattern Search:** Implement case-insensitive text transformations (.lower()) so variations in capitalization do not cause search errors.
- **Unique Identification Tracks:** Assign unique serial codes (like ISBN codes) to each item to manage duplicate book names correctly.

15. References

- Python Software Foundation. *Data Structures - Documentation on Lists*.
<https://docs.python.org/3/tutorial/datastructures.html>
- VITyarthi. *General Project Instructions & Submission Guidelines*.